

3. 動画投稿・処理・閲覧編

対象範囲

実装する機能

1. 動画投稿開始
 2. 動画の直接アップロード
 3. アップロード完了通知
 4. アップロード期限切れ管理
 5. 動画の実ファイル検証
 6. 動画変換
 7. サムネイル生成
 8. 動画処理の再試行
 9. Worker異常終了後のJob回収
 10. 変換失敗管理
 11. 自動公開
 12. リール閲覧・音声ON/OFF
 13. 動画詳細確認
 14. 投稿者ごとの公開動画一覧
 15. お気に入りに保存
 16. 保存解除
 17. 保存一覧
 18. 自分の投稿一覧
 19. 自分の投稿詳細
 20. 投稿者による非公開
 21. 投稿者による再公開
 22. 投稿削除
 23. Storage上の動画ファイル削除
 24. Frontend画面
 25. OpenAPI
 26. Backend、Worker、Frontend、結合テスト
 27. ブラウザ確認
-

前提条件

本編の実装前に、次が完成していること。

前提	使用目的
User Entity	投稿者、Status、Roleを確認する
usersテーブル	投稿者情報を参照する
Access Token認証	投稿・保存・投稿管理APIを保護する
Auth Middleware	active Userだけを認証必須APIへ通す
Optional Auth Middlewareの方針	公開APIでログイン状態を任意取得する
Rate Limit	投稿開始・完了通知へ適用する
Request ID	API、Worker、Jobログを追跡する
共通APIエラー	動画APIも同じエラー形式を使う
AuthContext	Frontendでログイン状態を共有する
ProtectedRoute	投稿画面・自分の投稿画面を保護する
管理者・ユーザー管理編	停止Userの動画公開防止を横断接続する

要件上の機能順

要件定義に書かれた順番は次のとおりとする。

1. 動画投稿開始
2. 動画の検証
3. 動画の変換
4. 変換失敗時の管理
5. 動画のアップロード
6. アップロード完了通知
7. サムネイル生成
8. 投稿の公開
9. リール閲覧
10. 動画詳細確認
11. 投稿者ごとの公開動画一覧
12. お気に入りに保存
13. 保存一覧
14. 投稿の削除
15. 動画ファイルの削除
16. 投稿者による非公開
17. 投稿ステータス確認

ただし、実際には動画本体がStorageへ保存されるまでFFprobeによる検証やFFmpegによる変換は実行できない。

そのため、仕様書上の機能順は維持しながら、コードと実行処理は依存関係に従って並べる。

実際のコード実装順

Backend・Worker

1. Video関連ファイルを配置する。
2. `CategoryCode` とカテゴリー一定数を実装する。
3. Video、SourceVideoMeta、OutputVideoMetaを実装する。
4. VideoProcessingJobを実装する。
5. SavedVideoを実装する。
6. IdempotencyRecordを実装する。
7. StorageCleanupJobを実装する。
8. Domain Errorを追加する。
9. Go Migrationを作成する。
10. VideoRepository Interfaceを定義する。
11. VideoProcessingJobRepository Interfaceを定義する。
12. SavedVideoRepository Interfaceを定義する。
13. StorageCleanupJobRepository Interfaceを定義する。
14. ObjectStorageRepository Interfaceを定義する。
15. MediaRepository Interfaceを定義する。
16. 各Repositoryを実装する。
17. VideoUsecaseを実装する。
18. VideoProcessingUsecaseを実装する。
19. VideoProcessingUsecaseへ動画処理Stageごとの処理時間計測を追加する。
20. SavedVideoUsecaseを実装する。
21. StorageCleanupUsecaseを実装する。
22. Upload期限切れ回収処理を実装する。
23. running状態の動画処理Job回収処理を実装する。
24. running状態のStorage削除Job回収処理を実装する。
25. 孤立Object検出処理を実装する。
26. 期限切れIdempotencyRecord削除処理を実装する。
27. VideoValidatorを実装する。

28. VideoControllerを実装する。
29. VideoControllerの公開動画一覧取得へ `author_id` Queryを接続し、指定された投稿者の公開動画だけを取得できるようにする。
30. SavedVideoControllerを実装する。
31. Optional Auth Middlewareを実装する。
32. 動画投稿用Rate Limitを既存構成へ追加する。
33. Routerへ動画APIを接続する。投稿者ごとの公開動画一覧も既存の `GET /videos` を使用し、`author_id` Queryで絞り込む。
34. API用 `main.go` へ依存関係を接続する。
35. Worker用 `main.go` へ依存関係を接続する。
36. Workerへ動画処理Job取得Loopを接続する。
37. WorkerへStorage削除Job取得Loopを接続する。
38. WorkerへUpload期限切れ回収Loopを接続する。
39. Workerへ異常終了Job回収Loopを接続する。
40. Workerへ孤立Object検出Loopを接続する。
41. Workerへ期限切れIdempotencyRecord削除Loopを接続する。
42. OpenAPIへ動画APIを追加し、`GET /videos` の任意Queryとして `author_id` と投稿者別公開一覧の取得条件を定義する。
43. BackendとWorkerのテストを実行する。
44. APIとWorkerを接続して状態遷移を確認する。
45. 定期処理を起動し、期限切れ・異常終了・孤立Objectを回収できることを確認する。

Frontend

1. 動画用Typeを作成する。
2. 動画API Clientを作成する。
3. 直接アップロード処理を作成する。
4. 動画投稿画面を作成する。
5. 自分の投稿一覧画面を作成する。
6. リール画面を作成する。
7. リール画面へ音声ON/OFF操作を接続する。
8. 動画詳細画面を作成する。
9. 投稿者ごとの公開動画一覧画面 `AuthorVideosPage` を作成する。
10. 保存一覧画面を作成する。
11. 保存操作の画面表記を「お気に入りに保存」とし、ヘッダーと同じ保存アイコンへ統一する

12. 非公開・再公開・削除操作を接続する。
13. React Routerへ各画面を接続する。 `/videos/author/:author_id` を投稿者ごとの公開動画一覧へ接続する。
14. Frontendテストを実行する。
15. PostgreSQL、Redis、Object Storage、API、Worker、Frontendを起動する。
16. ブラウザで投稿から削除まで確認する。
17. Frontendテストを実行する。
18. PostgreSQL、Redis、Object Storage、API、Worker、Frontendを起動する。
19. ブラウザで投稿から削除まで確認する。

完成状態

本編完成後、次をブラウザまたは結合テストで確認できること。

1. activeなUserだけが動画投稿を開始できる。
2. 投稿開始の連打でVideoが重複作成されない。
3. 動画本体がAPI Request Bodyを通らない。
4. Storageの保存先をクライアントが自由に指定できない。
5. アップロード完了通知の再送で処理Jobが増えない。
6. 期限切れ投稿が処理へ進まない。
7. 壊れた動画が処理へ進まない。
8. 拡張子を偽装した動画が処理へ進まない。
9. 10秒超過動画が公開されない。
10. 50MB超過動画が公開されない。
11. 9:16以外の動画が公開されない。
12. 入力解像度・fps条件違反が拒否される。
13. 条件内動画がWeb配信用の再生用MP4へ軽量化され、H.264、720×1280、30fps以下の公開用形式へ変換される。
14. 音声あり動画がAACになる。
15. 無音動画も公開できる。
16. サムネイルが生成される。
17. 処理成功時だけ `ready / published` になる。
18. 停止Userの動画は処理成功しても `ready / private` になる。
19. 最終失敗動画は `failed / private` になる。
20. 公開条件を満たす動画だけがリールに表示される。
21. リールの自動再生が同時に1件以内になる。

22. 表示中のリール動画について音声をON/OFFへ切り替えられる。
 23. 公開動画の投稿者を選択すると、その投稿者の `ready / published / 未削除` の動画だけを一覧表示できる。
 24. 同じ動画を重複保存できず、Frontendでは保存操作を「お気に入りに保存」と表示し、操作後の保存状態が反映される。
 25. 非公開・hidden・削除済み動画が保存一覧へ表示されない。
 26. 投稿者本人だけが非公開・再公開・削除できる。
 27. hidden動画を投稿者が再公開できない。
 28. 削除済み動画が一般取得APIへ出ない。
 29. 元動画、変換後動画、サムネイルが非同期削除される。
 30. Worker異常終了後もJobが永久に `running` で残らない。
 31. SQL、Stack Trace、署名付きURL、Storage Credentialをエラーへ含めない。
-

全体フロー

投稿開始から公開まで

1. Userが動画投稿画面を開く。
2. Frontendがタイトル、説明、カテゴリー、動画ファイルを受け取る。
3. Frontendが容量、拡張子、再生時間、縦横比を事前確認する。
4. Frontendが投稿操作単位の `Idempotency-Key` を生成する。
5. Frontendが投稿開始APIを呼ぶ。
6. Auth MiddlewareがAccess Token、User Status、TokenVersionを確認する。
7. Rate Limit Middlewareが投稿開始の制限を確認する。
8. ControllerがJSONと `Idempotency-Key` を受け取る。
9. Validatorが入力を検証・正規化する。
10. UsecaseがIdempotency KeyとRequest内容のHashを生成する。
11. Usecaseが元動画用Object Keyをサーバー側で生成する。
12. VideoとIdempotencyRecordを同一DB Transactionで保存する。
13. Object Storage用の期限付きUpload URLを発行する。
14. ControllerがVideo IDとUpload情報を返す。
15. FrontendがObject Storageへ動画本体を直接PUTする。
16. Upload成功後、Frontendが完了通知APIを呼ぶ。
17. Backendが指定済みObject Keyの存在をStorageへ確認する。
18. RepositoryがVideoを行ロックする。

19. `uploading / private` から `uploaded / private` へ変更する。
20. VideoProcessingJobを1件だけ作成する。
21. APIは変換完了を待たず `202 Accepted` を返す。
22. Workerが実行可能Jobを排他的に取得する。
23. Jobを `running` へ、Videoを `processing / private` へ変更する。
24. Workerが元動画を一時作業ディレクトリへ取得する。
25. FFprobeで実ファイルを解析する。
26. SourceVideoMetaを生成する。
27. FFmpegでWeb配信用の再生用MP4へ変換し、H.264、720×1280、30fps以下へ統一するとともに軽量化する。音声Trackが存在する場合はAACへ変換し、存在しない場合は映像のみのMP4を生成する。
28. FFmpegでサムネイルを生成する。
29. 変換後動画を再度FFprobeで確認する。
30. 変換後動画とサムネイルをStorageへ保存する。
31. RepositoryがJob、Video、Userの順番で行ロックする。
32. OutputVideoMetaを保存する。
33. 投稿者がactiveなら `ready / published` へ変更する。
34. 投稿者がsuspendedなら `ready / private` へ変更する。
35. Jobを `succeeded` へ変更する。
36. Frontendが投稿状況APIで完了状態を確認する。

変換失敗時

1. Workerが失敗を一時的障害か恒久的障害へ分類する。
2. 一時的障害かつ試行上限未到達の場合は `retry_wait` へ変更する。
3. `AvailableAt` へ次回実行可能日時を設定する。
4. 待機時間は試行回数に応じて段階的に延長する。
5. 恒久的障害または試行上限到達時はJobを `failed` へ変更する。
6. Videoを `failed / private` へ変更する。
7. 途中生成された変換後動画・サムネイルのCleanup Jobを作成する。
8. 一般取得APIへ動画を出さない。
9. 投稿者向けAPIでは処理失敗であることだけを返す。
10. SQLや内部コマンド内容を利用者へ返さない。

動画変換は初回1回と最大3回の再試行を合わせ、最大4回実行する。形式不正、破損、上限超過などの恒久的失敗は再試行しない。

投稿削除時

1. 投稿者が削除操作を実行する。
2. Auth MiddlewareがUserを確認する。
3. Repositoryが対象Videoを行ロックする。
4. `video.user_id` と認証User IDを比較する。
5. DeletedAtを設定する。
6. PublishStatusを `private` へ変更する。
7. `queued・retry_wait`の処理Jobを `cancelled` へ変更する。
8. 元動画のCleanup Jobを作成する。
9. Output Metaが存在する場合は、変換後動画とサムネイルのCleanup Jobを作成する。
10. Video論理削除とCleanup Job作成を同じDB Transactionで確定する。
11. APIはStorage削除完了を待たず応答する。
12. Cleanup Workerが各Objectを削除する。

running中に削除された場合、Workerは最終保存TransactionでDeletedAtを再確認する。削除済みであれば公開せず、生成物のCleanup Jobを作成してJobをcancelledへ変更する。

Clean Architectureの責務

層	責務
Entity	動画、Job、保存、冪等性、Cleanupの状態と不変条件を保持する
DB	投稿情報、状態、Object Key、Job、保存関係を永続化する
Repository	GORM、行ロック、Transaction、Storage SDK、外部コマンド境界を扱う
Usecase	投稿、動画処理、公開、閲覧、投稿者別公開一覧、保存、削除の業務フローを組み立てる
Validator	Path、Header、Query、JSONの形式を検証・正規化する
Controller	Echo Context、HTTP入力、HTTPレスポンスを扱う
Middleware	Auth、Optional Auth、Rate Limit、Body Limitを扱う
Router	URL、HTTPメソッド、Middleware、Controllerを接続する
Worker	Job取得、FFprobe、FFmpeg、Storage削除を非同期実行する
Frontend API	API通信と直接Uploadを扱う
React Page	投稿、リール、詳細、投稿者ごとの公開動画一覧、保存一覧、自分の投稿画面を表示する

Transactionの方針

汎用的な `TxManager` は作成しない。

Transactionが必要な処理は、用途が明確なRepositoryメソッド内で完結させる。

処理	Transaction内で行うこと
投稿開始	Video作成、IdempotencyRecord作成
Upload完了	Video行ロック、 uploaded 化、Processing Job作成
初回処理開始	Job排他取得、Jobを running へ変更、AttemptCount増加、Videoを processing へ変更
再試行開始	Job排他取得、Jobを running へ変更、AttemptCount増加。Videoは processing のまま維持
入力動画検証成功	Video行ロック、SourceMetaが未設定の場合だけSourceMeta保存
再試行待ち	Jobを retry_wait へ変更、AvailableAt更新。Videoは processing / private のまま維持
処理成功	OutputMeta保存、Job成功、Videoを ready へ変更、User Statusによる公開状態決定
最終失敗	Job失敗、Videoを failed / private へ変更、Cleanup Job作成
保存	公開Video確認、SavedVideo作成
投稿者非公開	Video行ロック、所有者確認、 private 化
投稿者再公開	UserとVideoを確認し、 published 化
投稿削除	Video論理削除、未開始Job取消、Cleanup Job作成
Upload期限切れ	Video行ロック、 expired 化、必要なCleanup Job作成
running Job回収	JobとVideoを行ロックし、再試行または最終失敗へ変更
Cleanup Job回収	Cleanup Jobを行ロックし、再試行または最終失敗へ変更

SourceMetaの保存規則

SourceMetaは、FFprobeによる入力動画検証が初めて成功した時点で保存する。

処理成功TransactionではSourceMetaを再保存しない。

再試行時は次のように扱う。

- SourceMetaが未設定の場合は、FFprobeを実行し、検証成功後に保存する。
- SourceMetaが設定済みの場合は、同じSourceMetaを再作成しない。
- SourceMeta設定後にFFmpegやStorage処理が一時失敗した場合は、既存SourceMetaを利用して再試行する。
- OutputMetaは、動画変換、サムネイル生成、出力動画検証がすべて成功した後だけ保存する。

Video Entityの状態遷移では、FFprobe成功後にSourceMetaを設定し、その後もProcessingStatusは **processing** のまま維持

DBとObject Storageの整合性

PostgreSQLとObject Storageを同じTransactionでCommitすることはできない。

そのため次の方針を採用する。

状況	処理
Video作成後にUpload URL発行失敗	同じ冪等性キーの再送で既存VideoへURLを再発行できる
Upload後に完了通知が来ない	期限切れ処理でVideoをexpiredにする
Upload期限切れ後にObjectが存在	Cleanup Jobを作成する
変換後動画Upload後にDB更新失敗	補償用Cleanup Jobを作成する
サムネイルUploadだけ失敗	先に保存した変換後動画のCleanup Jobを作成する
Cleanup Job作成自体が失敗	構造化エラーを記録し、孤立Object検出対象とする
Storage削除対象が既にある	削除成功扱いにする

動画とDBの分離、冪等性、補償削除、Workerの重複実行防止は要件定義の整合性要件である。

Object Storage Providerの確定

環境	Provider
ローカル開発・テスト	MinIO
本番	Cloudflare R2 Standard
API互換	S3互換API
Go SDK	AWS SDK for Go v2
Bucket公開	非公開
Upload	Presigned PUT
再生・サムネイル取得	Presigned GET
Backend・Worker操作	S3 Client
Object本体のDB保存	禁止

R2はS3互換Endpointを提供し、AWS SDKをEndpoint変更で使用できます。署名付きURLはPUT・GET・HEAD・DELETEへ対応し、ブラウザ利用にはBucket CORS設定が必要です。

MinIOもS3互換でDocker開発環境へ配置できるため、本番R2と同じRepository実装を利用できる。

Object Storage固定設定

Provider

本番環境はCloudflare R2 Standardを使用する。

ローカル開発環境と自動テストではMinIOを使用する。

BackendとWorkerはAWS SDK for Go v2のS3 Clientを使用し、環境ごとにEndpointとCredentialを差し替える。

Provider固有型をUsecaseへ渡さない。

Bucket

環境	Bucket
Local	coffee-reel-local
Test	coffee-reel-test
Production	coffee-reel-prod

1つのBucketを複数環境で共用しない。

Bucketは公開しない。

r2.dev 等の公開URLを動画Responseへ使用しない。

Endpoint

環境	Endpoint
Local	http://minio:9000
Production	https://{ACCOUNT_ID}.r2.cloudflarestorage.com

設定	Local	Production
Region	us-east-1	auto
ForcePathStyle	true	false
TLS	false	true

署名URL有効期間

URL	有効期間
元動画Upload URL	15分
変換後動画Read URL	10分
サムネイルRead URL	10分

署名URLをDBへ保存しない。

署名URLをアクセスログ、Workerログ、Frontend Consoleへ出力しない。

署名URLはBearer Tokenと同様に扱い、有効期限内はURLを知る者が操作できるため、短期間に限定する。

Upload制限

Presigned PUTへ次を署名対象として含める。

- Bucket
- Object Key
- Content-Type
- 有効期限

FrontendはBackendが返した Content-Type と同じ値を送信する。

Frontendから次を送らない。

- Access Token
- Refresh Token Cookie
- CSRF Cookie
- Storage Credential
- 任意のObject Key

Object Key

Object KeyはBackendだけが生成する。

```
videos/{video_id}/source/{random_id}.{ext}
videos/{video_id}/output/{random_id}.mp4
videos/{video_id}/thumbnail/{random_id}.jpg
```

項目	仕様
管理Prefix	videos/
Video ID	DB確定後のVideo ID
random_id	暗号学的乱数またはUUID
元ファイル名	使用しない
User入力	Keyへ直接含めない
Email・名前	Keyへ含めない
Path Traversal文字列	受け取らない

Object KeyをFrontendへ単独では返さない。

Frontendには署名付きURLと必要Headerだけを返す。

R2 CORS

Production Bucketには次の方針を設定する。

項目	設定
AllowedOrigins	本番Frontend URLだけ
AllowedMethods	PUT、GET、HEAD
AllowedHeaders	Content-Type
ExposeHeaders	ETag
MaxAgeSeconds	3600
* Origin	使用しない
Credentials	Storage Uploadでは送信しない

Local MinIOでは次を許可する。

```
http://localhost:3000
```

本番では実際のVercel Frontend URLだけを登録する。

ブラウザからPresigned URLを利用する場合もCORS設定が必要です。

環境変数

認証・Cookie・共通Rate Limit

認証、Cookie、CSRF、共通Body Limit、認証系Rate Limitの設定は「ユーザー認証編」を正本とし、本編では重複定義しない。

本編では動画投稿開始、Upload完了通知など、動画機能で追加するRate Limitと動画固有設定だけを定義する。

ファイル構成

Backend

```
backend/
├── entity/
│   ├── category.go
│   ├── video.go
│   ├── video_processing_job.go
│   ├── saved_video.go
│   ├── idempotency_record.go
│   ├── storage_cleanup_job.go
│   └── errors.go
├── db/
│   └── db.go
├── migrate/
│   ├── users.go
│   ├── refresh_tokens.go
│   ├── admin_audit_logs.go
│   ├── videos.go
│   ├── video_source metas.go
│   ├── video_output metas.go
│   ├── video_processing_jobs.go
│   ├── saved_videos.go
│   ├── idempotency_records.go
│   └── storage_cleanup_jobs.go
├── repository/
│   ├── user_repository.go
│   ├── video_repository.go
│   ├── video_processing_job_repository.go
│   ├── saved_video_repository.go
│   ├── storage_cleanup_job_repository.go
│   ├── object_storage_repository.go
│   └── media_repository.go
├── usecase/
│   ├── video_usecase.go
│   ├── video_processing_usecase.go
│   ├── saved_video_usecase.go
│   └── storage_cleanup_usecase.go
```

```

├── validator/
│   └── video_validator.go
├── controller/
│   ├── video_controller.go
│   └── saved_video_controller.go
├── middleware/
│   ├── auth_middleware.go
│   ├── optional_auth_middleware.go
│   └── ratelimit_middleware.go
├── worker/
│   └── worker.go
├── router/
│   └── router.go
├── cmd/
│   └── worker/
│       └── main.go
└── main.go

```

MigrationはすべてGoで実装する。通常のAPI起動時に無条件実行しない。

Frontend

```

frontend/
├── src/
│   ├── api/
│   │   ├── client.ts
│   │   ├── video.ts
│   │   └── upload.ts
│   ├── components/
│   │   ├── ReelVideo.tsx
│   │   ├── VideoCard.tsx
│   │   ├── VideoStatusBadge.tsx
│   │   └── UploadProgress.tsx
│   ├── pages/
│   │   ├── ReelPage.tsx
│   │   ├── VideoDetailPage.tsx
│   │   ├── AuthorVideosPage.tsx
│   │   ├── VideoUploadPage.tsx
│   │   ├── MyVideosPage.tsx
│   │   └── SavedVideosPage.tsx
│   ├── router/
│   │   └── AppRouter.tsx
│   ├── types/
│   │   └── video.ts
│   └── App.tsx

```

同じ画面でしか使用しない処理を不要に別Hookへ分割しない。

Entity仕様

CategoryCode

値	表示名
brewing	抽出
roasting	焙煎
latte_art	ラテアート
beans	コーヒー豆
equipment	器具

Requestから上記以外を受け取らない。

Video

役割

投稿者、投稿情報、元動画Object Key、Upload期限、処理状態、公開状態、論理削除状態を管理する。

フィールド

フィールド	仕様
ID	Video識別子
UserID	投稿者User ID
Category	CategoryCode
Title	trim後1~100文字
Description	0~1000文字。未入力時は空文字
OriginalObjectKey	サーバー生成の元動画Object Key
UploadExpiresAt	Upload可能期限
SourceMeta	FFprobe検証成功後に設定
OutputMeta	変換・サムネイル生成成功後に設定
ProcessingStatus	動画処理状態
PublishStatus	公開状態
CreatedAt	投稿開始日時
UpdatedAt	最終更新日時
DeletedAt	未削除時nil、削除時に日時を設定

処理状態

値	内容
uploading	Upload中
expired	Upload期限切れ

値	内容
uploaded	Upload完了
processing	検証・変換・サムネイル生成中
ready	動画処理完了
failed	動画処理最終失敗

公開状態

値	内容
private	一般非公開
published	一般公開
hidden	管理者による非公開

初期状態

- ProcessingStatusは uploading
- PublishStatusは private
- SourceMetaはnil
- OutputMetaはnil
- DeletedAtはnil

状態変更

操作	事前条件	結果
CompleteUpload	uploading、private、期限内、Object存在確認済み	uploadedへ変更
ExpireUpload	uploading、private、期限到達	expiredへ変更
StartProcessing	uploaded、private、未削除	processingへ変更
RecordSourceValidation	processing、SourceMeta未設定	SourceMetaを設定
CompleteProcessing	processing、SourceMetaあり、未削除	ready。User activeならpublished、それ以外はprivate
FailProcessing	processing、未削除	failed、private
SetPrivateByOwner	投稿者本人、ready、published	private
RepublishByOwner	投稿者本人、ready、private、User active	published
DeleteByOwner	投稿者本人、未削除	DeletedAt設定、private
CanBeViewedPublicly	なし	公開条件を満たす場合のみtrue

expired、failed、削除済みVideoは同じVideo上で再Uploadしない。再投稿は新しいVideoを作成する。Videoの状態遷移と公開条件は要件定義の状態表へ合わせる。

SourceVideoMeta

SourceMetaはFFprobe成功後に一度保存し、再試行時は次のように扱う。

1. SourceMetaが未設定ならFFprobe結果を保存する。
2. SourceMetaが既にあれば再作成しない。
3. 再試行では既存SourceMetaを使用して変換処理を再開する。
4. OutputMetaは変換・サムネイル・出力Probe成功後だけ保存する。
5. 処理成功Transactionでは、OutputMeta、Job成功、Video ready化、公開状態を確定する。

項目	条件
MIMEType	MP4は video/mp4 、MOVは video/quicktime
Container	mp4 または mov
SizeBytes	1～50,000,000 bytes
DurationMillis	1～10,000ms
Width	1～1080
Height	1～1920
Aspect	回転情報適用後に $Width \times 16 = Height \times 9$
FrameRate	0より大きく60以下
VideoCodec	空文字不可
Video Track	必須
Audio Track	任意
HasAudioがfalse	AudioCodecは空文字

Client申告Content-Type、拡張子、HTML File情報だけでSourceVideoMetaを生成しない。

OutputVideoMeta

項目	条件
VideoObjectKey	サーバー生成、空文字不可
ThumbnailObjectKey	サーバー生成、空文字不可
Container	mp4
Width	720
Height	1280
FrameRate	0より大きく30以下
VideoCodec	h264
HasAudioがtrue	AudioCodecは aac
HasAudioがfalse	AudioCodecは空文字
無音動画	映像のみのMP4として公開可能

VideoProcessingJob

状態

値	内容
queued	初回実行待ち
running	Worker実行中
retry_wait	再試行待ち
succeeded	処理成功
failed	最終失敗
cancelled	投稿削除等による取消

フィールド

フィールド	仕様
ID	Job識別子
VideoID	対象Video。1Videoにつき1件
Status	Job状態
AttemptCount	実行開始回数。初期値0
MaxAttempts	4
AvailableAt	次回実行可能日時
StartedAt	最後の実行開始日時
FinishedAt	成功、最終失敗、取消日時
LastErrorCode	安全な内部エラーコード
LastErrorMessage	秘密情報を除いた内部説明
CreatedAt	Job作成日時
UpdatedAt	最終更新日時

試行回数

タイミング	AttemptCount
作成時	0
初回開始	1
1回目の再試行	2
2回目の再試行	3
3回目の再試行	4

SavedVideo

項目	仕様
UserIDとVideoID	組み合わせを一意にする
新規保存	active Userかつ公開Videoだけ
保存解除	関係行を物理削除する
Video非公開後	関係行は保持してよい
保存一覧	現在も公開条件を満たすVideoだけ返す

IdempotencyRecord

動画投稿開始APIへ同じIdempotency-Keyが再送された場合に、重複したVideoを作らず、最初に作成したVideoへ紐づける。

アップロード完了通知の重複防止には使用しない。

アップロード完了通知は、Videoの状態確認、Video行ロック、VideoProcessingJobのVideoID一意保証によって重複実行を防ぐ。

フィールド

項目	仕様
ID	IdempotencyRecordを識別するID
UserID	投稿開始APIを実行したUser ID
Scope	video_create
KeyHash	Idempotency-Keyを専用HMAC鍵でHash化した値
RequestHash	正規化済み投稿入力のHash
ResourceID	最初に作成したVideo ID
ExpiresAt	冪等性記録を保持する期限
CreatedAt	冪等性記録の作成日時

不変条件

- UserID、Scope、KeyHashの組み合わせを一意にする。
- 平文Idempotency-KeyをDBへ保存しない。
- 平文Idempotency-Keyをログへ記録しない。
- 同じKeyHashかつ同じRequestHashの場合は、既存ResourceIDを返す。
- 同じKeyHashでRequestHashが異なる場合は、冪等性キー競合として拒否する。
- Video作成とIdempotencyRecord作成を同じDB Transactionで行う。
- ExpiresAtはCreatedAtより後とする。
- ExpiresAtを過ぎたRecordは定期削除対象とする。

保持期限

IdempotencyRecordの保持期間は設定値としてUsecaseへ注入する。

保持期間をコードへ直接記載しない。

IdempotencyRecord保持期間が未設定または0以下の場合は、APIの起動時に設定エラーとする。WorkerはDBへ保存済みのExpiresAtを使用して期限切れRecordを削除するため、保持期間自体を必要としない。

StorageCleanupJob

フィールド

フィールド	仕様
ID	Cleanup Jobを識別するID
VideoID	関連Video ID。孤立Objectの場合はnil
ObjectKey	削除対象のObject Key
AssetType	<code>original</code> 、 <code>processed</code> 、 <code>thumbnail</code> 、 <code>unknown</code>
Cause	Cleanup Jobを作成した理由
Status	削除処理状態
AttemptCount	削除処理を開始した回数。初期値0
MaxAttempts	初回を含む最大実行回数。4
AvailableAt	次回実行可能日時
StartedAt	最後に削除処理を開始した日時。未実行時はnil
FinishedAt	成功または最終失敗日時
LastErrorCode	秘密情報を含まない安全なエラーコード
LastErrorMessage	調査用の安全な内部説明
CreatedAt	Job作成日時
UpdatedAt	最終更新日時

不変条件

- ObjectKeyを空文字にしない。
- 同じObject Keyの未完了Cleanup Jobを重複作成しない。
- AttemptCountはMaxAttemptsを超えない。
- Storage上に対象Objectが存在しない場合も成功扱いにする。
- 署名付きURLやStorage Credentialを保存しない。
- 動画削除などのDB更新とCleanup Job作成を同じTransactionで行う。
- `failed` となったJobを削除せず、管理者確認対象として保持する。

状態

値	内容
<code>queued</code>	削除待ち
<code>running</code>	削除中
<code>retry_wait</code>	再試行待ち
<code>succeeded</code>	削除成功または対象なし
<code>failed</code>	最終失敗

AssetType

- `original`
- `processed`

- `thumbnail`
- `unknown`

Cause

- `video_delete`
- `upload_expired`
- `process_failed`
- `rollback_cleanup`
- `orphan_detected`

初回1回と最大3回の再試行を合わせ、MaxAttemptsは4とする。

DB仕様

使用するテーブル

テーブル	用途
<code>videos</code>	投稿情報、Object Key、Upload期限、処理・公開・削除状態
<code>video_source metas</code>	FFprobeで検証した元動画情報
<code>video_output metas</code>	変換後動画とサムネイル情報
<code>video_processing_jobs</code>	動画処理Job
<code>saved_videos</code>	Userと保存Videoの関係
<code>idempotency_records</code>	投稿開始の重複防止
<code>storage_cleanup_jobs</code>	不要Objectの削除Job
<code>users</code>	投稿者Status確認
<code>admin_audit_logs</code>	停止Userの動画hidden化で使用

動画関連テーブルの正本は、本編のEntity仕様、DB仕様、Migration仕様とする。
 通常のテーブル定義は、各EntityのGORMタグとGORM Migratorから作成する。
 主キー、NOT NULL、UNIQUE、外部キー、Indexなど、本編で明示したDB保証をMigrationへ反映する。

GORMタグだけでは表現しにくいPostgreSQL固有機能が必要な場合は、その機能を使用する編でMigration方法を明示する。

必須のDB保証

- 1VideoにつきProcessing Jobは1件
- UserIDとVideoIDの保存関係は1件
- UserID、Scope、KeyHashの冪等性記録は1件
- 同一Object Keyの未完了Cleanup Jobを重複作成しない
- Videoの状態値を定義済み値へ限定する

- JobのAttemptCountがMaxAttemptsを超えない
- DeletedAt設定済みVideoを一般一覧へ含めない

一覧の並び順

一覧	並び順
公開リール	Video CreatedAt降順、ID降順
自分の投稿	Video CreatedAt降順、ID降順
保存一覧	SavedVideo CreatedAt降順、ID降順
Worker Job	AvailableAt昇順、ID昇順
Cleanup Job	AvailableAt昇順、ID昇順

Repository仕様

共通方針

- InterfaceをUsecaseへ注入する
- GORMはRepository内だけで扱う
- User入力をSQLへ直接連結しない
- ORDER BYは固定する
- `any`、`map[string]interface{}` を使用しない
- 汎用TxManagerを作成しない
- DB制約違反をDomain Errorへ変換する
- SQLSTATEや制約名をAPIへ返さない

VideoRepository

メソッド	責務
CreateWithIdempotency	Videoと冪等性記録を同一Transactionで作成する
CompleteUpload	Videoを行ロックし、uploaded化とJob作成を行う
FindPublicById	公開条件を満たすVideoを取得する
ListPublic	公開リールをCursor方式で取得する
FindOwnedById	投稿者本人用詳細を取得する
ListOwned	自分の投稿一覧を取得する
SetPrivateByOwner	所有者確認後にprivate化する
RepublishByOwner	User active確認後にpublished化する
DeleteByOwner	論理削除、Job取消、Cleanup Job作成を行う
ExpireUploads	期限切れVideoをexpired化する
CompleteProcessing	処理成功結果を原子的に保存する

メソッド	責務
FailProcessing	最終失敗とCleanup Jobを原子的に保存する
RecordSourceValidation	SourceMetaが未設定の場合だけ、FFprobe検証結果を保存する
IsObjectReferenced	Object KeyがVideo、SourceMeta、OutputMetaのいずれかから参照されているか確認する
DeleteExpiredIdempotencyRecords	保持期限を過ぎたIdempotencyRecordを削除する

VideoProcessingJobRepository

メソッド	責務
ClaimNext	実行可能Jobを <code>FOR UPDATE SKIP LOCKED</code> で取得する
ScheduleRetry	retry_waitへ変更する
Cancel	cancelledへ変更する
RecoverTimedOut	期限超過running Jobを再試行または失敗へ変更する

処理	使用するRepositoryメソッド
処理成功	<code>VideoRepository.CompleteProcessing</code>
最終失敗	<code>VideoRepository.FailProcessing</code>
一時失敗	<code>VideoProcessingJobRepository.ScheduleRetry</code>
投稿削除による取消	<code>VideoProcessingJobRepository.Cancel</code> または <code>VideoRepository.DeleteByOwner</code> 内の取消処理

Job ClaimのTransactionは短くする。

Jobを `running` へ変更したまま、FFmpeg終了までDB Transactionを保持しない。

ClaimNext

実行可能なStorage Cleanup Jobを排他的に取得し、同じ短いTransaction内で実行開始状態へ変更する。

1. `queued`、または実行可能日時を迎えた `retry_wait` のJobを `FOR UPDATE SKIP LOCKED` で1件取得する。
2. AttemptCountがMaxAttempts未満であることを確認する。
3. Jobを `running` へ変更する。
4. AttemptCountを1増加する。
5. StartedAtを現在時刻へ更新する。
6. FinishedAtをnilのまま維持する。
7. TransactionをCommitする。
8. Commit後にObject Storageの削除処理を開始する。
9. Storage削除完了までDB Transactionを保持しない。

対象Job

- Statusが `queued`
- またはStatusが `retry_wait` かつAvailableAtが現在時刻以前
- AttemptCountがMaxAttempts未満

Transaction内の処理

1. 対象Jobを `FOR UPDATE SKIP LOCKED` で1件取得する。
2. Jobが現在も実行可能状態であることを確認する。
3. Jobを `running` へ変更する。
4. AttemptCountを1増加する。
5. StartedAtを現在時刻へ更新する。
6. FinishedAtをnilのまま維持する。
7. TransactionをCommitする。
8. Commit後に元動画の取得、FFprobe、FFmpeg処理を開始する。動画処理完了までDB Transactionを保持しない。

Storage削除完了までDB Transactionを保持しない。

Jobを `running` へ変更したまま、FFmpeg終了までDB Transactionを保持しない。

RecoverTimedOut

1. `running` かつStartedAtから処理Timeoutを超えたJobを行ロックする。
2. 対象Videoを行ロックする。
3. Jobが現在も `running` であることを再確認する。
4. Videoが削除済みの場合はJobを `cancelled` へ変更する。
5. AttemptCountがMaxAttempts未満の場合はJobを `retry_wait` へ変更する。
6. 再試行時はAvailableAtへ次回実行可能日時を設定する。
7. 再試行時のVideoは `processing / private` のまま維持する。
8. AttemptCountがMaxAttempts以上の場合はJobを `failed` へ変更する。
9. 最終失敗時はVideoを `failed / private` へ変更する。
10. 削除対象の生成物がある場合はCleanup Jobを作成する。
11. すべてを同じTransactionで確定する。

SavedVideoRepository

メソッド	責務
Save	公開Video確認後に保存関係を作る

メソッド	責務
Remove	UserIDとVideoIDに一致する関係を削除する
ListByUser	公開条件を満たす保存Videoだけ取得する
Exists	ログインUserが保存済みか確認する

同じ保存Requestの再送は、既に保存済みでも成功扱いにできる。

ObjectStorageRepository

メソッド	責務
CreateUploadURL	指定Object Key用の期限付きUpload URLを発行する
Exists	Objectの存在を確認する
Stat	サイズ、Content-Type等を取得する
Download	Worker作業領域へ取得する
UploadProcessed	変換後動画を保存する
UploadThumbnail	サムネイルを保存する
CreateReadURL	許可済みObjectの期限付き取得URLを発行する
Delete	Objectを削除する
ListManagedObjects	Coffee Reelが管理するStorage Prefix配下のObjectをCursor方式で取得する

Storage SDK固有型をUsecaseへ返さない。

`ListManagedObjects` はStorage SDK固有型をUsecaseへ返さず、次を持つ固定構造体を返す。

ManagedObject

項目	内容
ObjectKey	Storage上のObject Key
SizeBytes	Object容量
LastModifiedAt	Storage上の最終更新日時

ManagedObjectPage

項目	内容
Items	ManagedObjectの一覧
NextCursor	次ページ取得用Cursor。最終ページではnil
HasMore	次ページが存在する場合はtrue

`ListManagedObjects` は、Storage SDK固有のObject型やCursor型をUsecaseへ返さず、`ManagedObjectPage` を返す。

`DetectOrphanObjects` は `HasMore` がtrueの間、`NextCursor` を使って次ページを取得する。

MediaRepository

メソッド	責務
Probe	サーバー側で元動画の先頭バイトから実MIMEを判定し、FFprobeでContainer、Track、時間、解像度、回転、fps、Codecを解析する
Transcode	投稿動画をWeb配信用の再生用MP4へ軽量化しながら変換し、H.264、720×1280、30fps以下へ統一する。音声Trackが存在する場合はAACへ変換し、存在しない場合は映像のみとする。
GenerateThumbnail	サムネイルを生成する
ProbeOutput	変換後動画を再検証する

Shell文字列を組み立てない。

`exec.CommandContext` へ実行ファイルと固定引数を個別に渡す。

タイトル、説明、元ファイル名、Object Keyをコマンド文字列へ連結しない。

StorageCleanupJobRepository

メソッド	責務
Create	Cleanup Jobを作成する
ClaimNext	実行可能Jobを排他的に取得する
ScheduleRetry	一時障害を再試行待ちへ変更する
MarkSucceeded	削除成功・対象なしを成功へ変更する
MarkFailed	最終失敗へ変更する
RecoverTimedOut	runningのまま残ったJobを回収する

実行可能なStorage Cleanup Jobを排他的に取得し、同じ短いTransaction内で実行開始状態へ変更する。

Usecase仕様

VideoUsecase

メソッド	内容
StartUpload	投稿情報作成とUpload URL発行
CompleteUpload	元動画存在確認とJob作成
ListReels	公開動画一覧を取得する。 <code>author_id</code> が指定された場合は同じ公開条件を維持したまま指定投稿者の動画へ絞り込む。
GetDetail	公開動画詳細取得
ListMine	自分の投稿一覧
GetMine	自分の投稿詳細
SetPrivate	投稿者非公開
Republish	投稿者再公開

メソッド	内容
Delete	投稿削除

StartUpload

1. active Userを受け取る。
2. 検証済みTitle、Description、Category、申告Content-Type、申告Sizeを受け取る。
3. Idempotency-Keyを受け取る。
4. 平文Keyを専用HMAC鍵でHash化する。
5. 正規化済みRequestを固定構造体へ格納する。
6. JSON化したbyte列からRequestHashを生成する。
7. Object Keyをサーバー側で生成する。
8. Upload期限を設定する。
9. Videoを `uploading / private` で生成する。
10. CreateWithIdempotencyを呼ぶ。
11. 同一Key・同一入力なら既存Videoを取得する。
12. 既存Videoがuploadingかつ期限内なら、新しい期限付きURLを発行できる。
13. expired、failed、deletedのVideoへUpload URLを再発行しない。
14. VideoとUpload情報を返す。

Upload URLの具体的な有効時間は現在の要件定義で固定されていないため、必須設定値として注入する。未設定時は起動エラーにする。コード内へ勝手な時間を直書きしない。

CompleteUpload

1. active UserとVideo IDを受け取る。
2. 対象Videoを取得する。
3. 投稿者本人か確認する。
4. UploadExpiresAtを確認する。
5. Upload期限切れの場合は、Repositoryの期限切れ処理を呼び出す。
6. サーバーが保持するOriginalObjectKeyを使用する。
7. ClientからObject Keyを受け取らない。
8. Storage上のObject存在を確認する。
9. Storage上のサイズとContent-Typeを確認する。
10. RepositoryのCompleteUploadを呼ぶ。
11. Videoを行ロックする。
12. uploadingの場合だけuploadedへ変更する。
13. Processing Jobがなければ1件作成する。

14. 既にJobがある場合は追加作成しない。
15. 現在状態を返す。

Upload期限切れの場合

1. Repositoryの期限切れ処理を呼び出す。
2. RepositoryはVideoを行ロックする。
3. 現在も `uploading / private` で期限切れであることを再確認する。
4. Videoを `expired / private` へ変更する。
5. OriginalObjectKeyを対象としたCleanup Jobを作成する。
6. Video更新とCleanup Job作成を同じTransactionで確定する。
7. Storageの存在確認は行わない。
8. `409 Conflict` を返して処理を終了する。
9. Processing Jobは作成しない。

期限内の場合だけ、OriginalObjectKeyを使ったStorage存在確認、サイズ確認、Content-Type確認へ進みます。

ListReels

- 公開条件をDB取得条件へ必ず含める
- OutputMetaが存在するVideoだけ返す
- `author_id` が指定された場合は `videos.user_id = author_id` を取得条件へ追加する
- `author_id` を指定しても `ready / published / deleted_at IS NULL` の公開条件を緩めない。
- Cursorには `author_id` を含む検索条件の識別情報を反映し、異なる投稿者の条件へCursorを使い回さない。
- 投稿者Name、Title、Description、Categoryを返す
- 期限付き再生URLとサムネイルURLを返す
- Object Keyを返さない
- Optional AuthでUserが存在する場合は `is_saved` を返せる
- hidden、private、deletedを返さない

GetDetail

一般詳細APIでは公開条件を満たすVideoだけ取得する。

存在するがprivate、hidden、deletedの場合も、一般利用者へ存在を推測させないため404として扱う。

ListMine • GetMine

投稿者本人には次を返す。

- ID

- Title
- Category
- ProcessingStatus
- PublishStatus
- CreatedAt
- UpdatedAt
- UploadExpiresAt
- 安全な失敗状態
- readyかつprivateの場合の期限付き再生情報

次を返さない。

- OriginalObjectKey
- VideoObjectKey
- ThumbnailObjectKey
- Storage Credential
- Worker内部Error Message
- FFmpeg引数
- SQL

hidden 動画は投稿者向け画面で状態を確認できるが、投稿者本人へ期限付き再生URL・サムネイルURLを返さない。管理者によってhiddenとなった動画は投稿者から再公開できない。

SetPrivate

1. active Userを確認する。
2. Videoを行ロックする。
3. 所有者を確認する。
4. ready / publishedであることを確認する。
5. privateへ変更する。
6. 保存関係は削除しない。
7. 一般一覧と保存一覧から除外する。

Republish

1. active Userを確認する。
2. Videoを行ロックする。
3. 所有者を確認する。
4. ready / privateであることを確認する。
5. deletedでないことを確認する。

6. publishedへ変更する。
7. hiddenの場合は拒否する。

Delete

1. active Userを確認する。
 2. Videoを行ロックする。
 3. 所有者を確認する。
 4. 未削除であることを確認する。
 5. DeleteByOwnerを実行する。
 6. Videoをprivate化しDeletedAtを設定する。
 7. 未開始Jobをcancelledへ変更する。
 8. 存在するObject KeyごとにCleanup Jobを作成する。
 9. Commit後すぐに204を返す。
 10. Storage削除完了をHTTP内で待たない。
-

VideoProcessingUsecase

ProcessNext

Jobを取得した時点から動画処理全体の経過時間を計測し、主要Stageごとの開始・完了・失敗と所要時間をログへ記録する。

1. VideoProcessingJobRepositoryの `ClaimNext` を呼び出す。
2. Claim対象がない場合は処理なしで終了する。
3. Jobごとの一時作業ディレクトリを作成する。
4. 動画処理用Contextへ5分のTimeoutを設定する。
5. VideoのOriginalObjectKeyを使用して元動画をStorageから取得する。
6. User入力のファイル名を一時ファイル名へ使用しない。
7. VideoのSourceMetaが未設定か確認する。
8. SourceMetaが未設定の場合はFFprobeを実行する。
9. FFprobe結果から実ファイル形式、動画時間、容量、解像度、縦横比、fps、映像Trackを検証する。
10. 検証成功時はVideoRepositoryの `RecordSourceValidation` を呼び出す。
11. SourceMetaが既に設定されている場合は、SourceMetaの二重保存を行わない。
12. FFmpegでWeb配信用の再生用MP4へ軽量化しながら変換し、H.264、720×1280、30fps以下へ統一する。
13. 音声Trackが存在する場合はAACへ変換する。

14. 音声Trackが存在しない場合は映像のみのMP4を生成する。
15. サムネイルを生成する。
16. 変換後動画へFFprobeを実行する。
17. OutputVideoMetaの条件を検証する。
18. 変換後動画をObject Storageへ保存する。
19. サムネイルをObject Storageへ保存する。
20. VideoRepositoryの `CompleteProcessing` を呼び出す。
21. CompleteProcessing内でJob、Video、Userを必要な順序で行ロックする。
22. Videoが削除済みでないことを再確認する。
23. OutputMetaを保存する。
24. Userがactiveの場合はVideoを `ready / published` へ変更する。
25. Userがsuspendedの場合はVideoを `ready / private` へ変更する。
26. Jobを `succeeded` へ変更する。
27. DB更新失敗時は、Storageへ保存済みの生成物をCleanup対象にする。
28. 成功・失敗・Timeout・Cancelのすべてで一時的作業ディレクトリを削除する。

FFprobe処理へ次を追加。

1. 元動画の先頭バイトからサーバー側でMIMEタイプを判定する。
2. FFprobeの `format_name` と `major_brand` を解析する。
3. QuickTime系を `mov` へ正規化する。
4. MP4系を `mp4` へ正規化する。
5. 判定不能なISO Base Media形式を推測で許可しない。
6. `mp4` は `video/mp4`、`mov` は `video/quicktime` へ正規化する。
7. ブラウザ申告MIMEやStorageのContent-Typeだけを最終判定に使用しない。
8. 実形式が許可外、解析不能、映像Trackなしの場合は恒久的失敗とする。

一時的な失敗

次のような一時的な失敗では、試行上限未到達の場合にJobを `retry_wait` へ変更する。

- Storageの一時的通信障害
- Storageの一時的な5xx
- Worker処理Timeout
- 一時的な外部プロセス実行失敗

Videoは `processing / private` のまま維持する。

恒久的な失敗

次は再試行せず、Jobを `failed`、Videoを `failed / private` へ変更する。

- 対応外の実ファイル形式
- 動画破損
- 動画時間超過
- ファイル容量超過
- 解像度超過
- 9:16以外
- fps超過
- 映像Trackなし

Video削除済み

動画処理中にVideoが削除済みであることを確認した場合は、動画処理失敗ではなく、投稿削除による取消として扱う。

1. Jobを `cancelled` へ変更する。
2. Videoを `failed` へ変更しない。
3. 利用者向け `failure_code` を設定しない。
4. 変換後動画またはサムネイルをStorageへ保存済みの場合はCleanup Jobを作成する。
5. SourceMetaは削除しない。
6. OutputMetaは保存しない。
7. 公開処理へ進めない。
8. JobのFinishedAtへ取消日時を設定する。

SavedVideoUsecase

Save

1. active Userを確認する。
2. 公開条件を満たすVideoを取得する。
3. 保存関係を作成する。
4. UNIQUE競合は保存済みとして扱う。
5. 成功結果を返す。

Remove

1. active Userを確認する。
2. UserIDとVideoIDが一致する保存関係を削除する。
3. 関係がない場合も再送可能な成功扱いにする。

List

1. active Userを確認する。
2. Userの保存関係を取得する。
3. 公開条件を満たすVideoだけ返す。
4. private、hidden、deletedは返さない。
5. 保存関係自体は自動削除しない。

StorageCleanupUsecase

1. StorageCleanupJobRepositoryのClaimNextを呼び出す。
2. Claim対象がない場合は処理なしで終了する。
3. ClaimNextによってJobは既にrunningへ変更済みとする。
4. ObjectStorageRepository.Deleteを呼ぶ。
5. Objectが存在しない場合も成功扱いにする。
6. 一時障害はretry_waitへ変更する。
7. 最大試行回数到達でfailedへ変更する。
8. failed Jobは管理者確認対象として保持する。

Worker定期処理

Workerは動画処理JobとStorage削除Jobの実行だけでなく、次の定期処理を行う。

Upload期限切れ回収

VideoProcessingUsecaseへ `ExpireUploads` を定義する。

1. ProcessingStatusが `uploading` で、UploadExpiresAtが現在時刻以前のVideoを取得する。
2. 対象Videoを行ロックする。
3. 現在も `uploading / private` であり、期限切れであることを再確認する。
4. Videoを `expired / private` へ変更する。
5. OriginalObjectKeyを対象としたCleanup Jobを作成する。
6. Storage上のObject存在確認は行わない。
7. Video更新とCleanup Job作成を同じDB Transactionで確定する。
8. Cleanup Workerは、対象Objectが存在しない場合も成功扱いにする。

動画処理Job異常回収

VideoProcessingUsecaseへ `RecoverTimedOutJobs` を定義する。

1. `running` のVideoProcessingJobを対象にする。

2. StartedAtから動画処理Timeoutを超えたJobを取得する。
3. JobとVideoを行ロックする。
4. 試行上限未到達なら `retry_wait` へ変更する。
5. 試行上限到達ならJobを `failed` へ変更する。
6. 最終失敗時はVideoを `failed / private` へ変更する。
7. 必要なCleanup Jobを作成する。

Storage削除Job異常回収

StorageCleanupUsecaseへ `RecoverTimedOutJobs` を定義する。

1. `running` のStorageCleanupJobを対象にする。
2. StartedAtからCleanup Timeoutを超えたJobを取得する。
3. Jobを行ロックする。
4. 試行上限未到達なら `retry_wait` へ変更する。
5. 試行上限到達なら `failed` へ変更する。

孤立Object検出

StorageCleanupUsecaseへ `DetectOrphanObjects` を定義する。

1. Object StorageのCoffee Reel管理Prefix配下をCursor方式で取得する。
2. Object作成直後の誤判定を防ぐため、設定された最小経過時間を超えたObjectだけを対象にする。
3. VideoRepositoryの `IsObjectReferenced` を呼び出す。
4. videos、video_source metas、video_output metasのいずれからも参照されていないObjectを孤立Objectと判定する。
5. 同じObject Keyの未完了Cleanup Jobが存在しないことを確認する。
6. Causeを `orphan_detected` としてCleanup Jobを作成する。
7. Storage Objectを検出処理内で直接削除しない。

期限切れIdempotencyRecord削除

VideoProcessingUsecaseへ `DeleteExpiredIdempotencyRecords` を定義する。

1. ExpiresAtが現在時刻以前のIdempotencyRecordを対象にする。
2. 一度に削除する件数へ上限を設ける。
3. 定期的に分割削除する。
4. 平文Idempotency-Keyは扱わない。

Worker Loop

Workerは次のLoopを独立して実行する。

- 動画処理Job取得Loop
- Storage Cleanup Job取得Loop
- Upload期限切れ回収Loop
- 動画処理Job異常回収Loop
- Storage Cleanup Job異常回収Loop
- 孤立Object検出Loop
- IdempotencyRecord期限切れ削除Loop

各Loopは設定された間隔で実行する。

エラー発生時に高速な無限ループへ入らないようにする。

一つのLoopの失敗でWorker全体を終了させない。

Validator仕様

VideoValidator

メソッド	検証対象
ValidateStartUpload	Title、Description、Category、申告Content-Type、申告Size
ValidateIdempotencyKey	Idempotency-Key
ValidateVideoID	Path Video ID
ValidateListQuery	limit、cursor、author_id

投稿開始入力

項目	条件
Title	必須、trim後1~100文字
Description	任意、0~1000文字
Category	定義済み5カテゴリー
Content-Type	video/mp4 または video/quicktime
Size	1~50,000,000 bytes
Idempotency-Key	必須、空白禁止、長さ上限を設定する
Role	Requestから受け取らない
UserID	Request Bodyから受け取らない
Object Key	Requestから受け取らない
ProcessingStatus	Requestから受け取らない
PublishStatus	Requestから受け取らない
author_id	任意。指定する場合は1以上かつ有効なUser ID形式

Client申告値は事前拒否とUpload権限制限に使用する。最終判定はStorage実体とFFprobe結果で行う。

Idempotency-Keyは、設定された最大長以下であることを確認する。

Video ID

- 数字である
- 1以上
- `math.MaxInt64` 以下
- 負数、0、上限超過を拒否する

Cursor

公開動画一覧、自分の投稿、保存一覧はCursor方式へ統一する。投稿者ごとの公開動画一覧も既存の公開動画一覧Cursorを使用し、`author_id` を含む取得条件を維持する。

項目	条件
limit未指定	20
limit	1~100
cursor未指定	最初のページ
不正Base64	400
不正JSON	400
不正日時・ID	400
<code>any</code>	使用しない
不明フィールド	拒否する

CursorはCreatedAtとIDを固定構造体へ格納し、JSON化後にBase64 Raw URL Encodingする。

`author_id` を指定した公開動画一覧では、Cursorを別の `author_id` 条件へ使い回さない。

保存一覧ではSavedVideoのCreatedAtとIDを使用する。

Controller仕様

VideoController

メソッド	内容
StartUpload	投稿開始Requestを受け取る
CompleteUpload	Upload完了通知を受け取る
ListReels	公開動画一覧を返す。任意の <code>author_id</code> Queryがある場合は検証後にUsecaseへ渡し、その投稿者の公開動画へ絞り込む。
Detail	公開動画詳細を返す
ListMine	自分の投稿一覧を返す

メソッド	内容
MineDetail	自分の投稿詳細を返す
SetPrivate	投稿者非公開を実行する
Republish	投稿者再公開を実行する
Delete	投稿論理削除を実行する

Controllerは次を行わない。

- Object Key生成
- Storage存在確認
- FFprobe、FFmpeg実行
- 所有者判定
- 状態遷移
- DB Transaction
- Cursor条件のSQL生成
- 署名URLの保存

SavedVideoController

メソッド	内容
Save	保存する
Remove	保存解除する
List	保存一覧を返す

Middleware仕様

Auth Middleware

次のAPIへ適用する。

- 動画投稿開始
- Upload完了通知
- 自分の投稿一覧・詳細
- 投稿者非公開
- 投稿者再公開
- 投稿削除
- 保存
- 保存解除
- 保存一覧

Userが `suspended` の場合、Controllerへ進めない。

Optional Auth Middleware

`GET /videos` の公開動画一覧・投稿者別公開一覧と、公開動画詳細へ適用する。

- Authorization Headerがない場合はゲストとして継続する
- Headerがある場合は通常のJWT検証を行う
- 不正なTokenが指定された場合はゲスト扱いにせず401を返す
- 正常なUserがいる場合だけ `is_saved` 判定へ使用する

Rate Limit

API・識別単位	Capacity	補充速度	Cost
投稿開始・User ID	3	60秒に1 Token	1
投稿開始・IP	10	10秒に1 Token	1
Upload完了通知・User ID	10	1秒に1 Token	1
Upload完了通知・IP	30	1秒に2 Token	1

投稿開始はUser IDとIPの両方を確認する。

Upload完了通知もUser IDとIPの両方を確認する。

どちらか一方でも制限超過なら、VideoUsecaseへ到達させない。

Body Limit

動画投稿開始と完了通知はJSONだけを受け取る。

動画本体はBody Limit対象のAPIへ送らない。

CSRF

動画APIはAuthorization HeaderのAccess Tokenを使用し、Refresh Token Cookieを認証に使用しないためCSRF Middlewareを適用しない。

Router・API仕様

HTTP	URL	内容	認証	Rate Limit
<code>POST</code>	<code>/videos</code>	動画投稿開始	Access Token	必要
<code>POST</code>	<code>/videos/:video_id/upload-complete</code>	Upload完了通知	Access Token	必要
<code>GET</code>	<code>/videos</code>	公開動画一覧・投稿者別公開動画一覧	Optional Auth	不要
<code>GET</code>	<code>/videos/:video_id</code>	公開動画詳細	Optional Auth	不要
<code>GET</code>	<code>/me/videos</code>	自分の投稿一覧	Access Token	不要

HTTP	URL	内容	認証	Rate Limit
GET	/me/videos/:video_id	自分の投稿詳細	Access Token	不要
PATCH	/me/videos/:video_id/private	投稿者非公開	Access Token	不要
PATCH	/me/videos/:video_id/publish	投稿者再公開	Access Token	不要
DELETE	/me/videos/:video_id	投稿削除	Access Token	不要
PUT	/videos/:video_id/saved	保存	Access Token	不要
DELETE	/videos/:video_id/saved	保存解除	Access Token	不要
GET	/me/saved-videos	保存一覧	Access Token	不要

Query	必須	内容
author_id	いいえ	指定した投稿者User IDの公開動画だけへ絞り込む。未指定時は従来どおり公開動画一覧を返す

API構成は投稿開始、完了通知、公開閲覧、自分の投稿管理、保存の責務へ分ける。投稿者ごとの公開動画一覧は公開閲覧の一部として既存 GET /videos の author_id Queryで扱い、重複した専用APIを作成しない。

API成功Status

HTTP	URL	正常時Status	Response
POST	/videos	201 Created	Video情報とUpload情報
POST	/videos/:video_id/upload-complete	202 Accepted	現在の処理状態
GET	/videos	200 OK	公開動画一覧または投稿者別公開動画一覧
GET	/videos/:video_id	200 OK	公開動画詳細
GET	/me/videos	200 OK	自分の投稿一覧
GET	/me/videos/:video_id	200 OK	自分の投稿詳細
PATCH	/me/videos/:video_id/private	200 OK	更新後のVideo状態
PATCH	/me/videos/:video_id/publish	200 OK	更新後のVideo状態
DELETE	/me/videos/:video_id	204 No Content	Bodyなし
PUT	/videos/:video_id/saved	204 No Content	Bodyなし
DELETE	/videos/:video_id/saved	204 No Content	Bodyなし
GET	/me/saved-videos	200 OK	保存動画一覧

幂等な再送

- 同じIdempotency-Keyと同じ入力による投稿開始再送では、新しいVideoを作成しない。
- 投稿開始再送時も同じResponse形式を返す。
- Upload完了通知の再送では、新しいProcessing Jobを作成しない。

- 保存済み動画への保存再送は `204 No Content` とする。
- 保存されていない動画への保存解除再送も `204 No Content` とする。

一覧Response

次の一覧APIは共通形式へ統一する。

- 公開動画一覧
- 自分の投稿一覧
- 保存一覧

```
{
  "items": [],
  "next_cursor": null,
  "has_more": false
}
```

API Request ・ Response

動画投稿開始

Header

Header	必須	内容
Authorization	必須	Bearer Access Token
Idempotency-Key	必須	投稿操作ごとの一意な値
Content-Type	必須	<code>application/json</code>

Request

```
{
  "title": "ハンドドリップの蒸らし方",
  "description": "蒸らし時間の違いを紹介します",
  "category": "brewing",
  "file_content_type": "video/mp4",
  "file_size_bytes": 12000000
}
```

Response

```
{
  "video": {
    "id": 125,
    "title": "ハンドドリップの蒸らし方",
    "description": "蒸らし時間の違いを紹介します",
    "category": "brewing",
    "processing_status": "uploading",
    "publish_status": "private",
  }
}
```



```

      "upload_expires_at": "2026-07-16T12:00:00Z",
      "created_at": "2026-07-16T11:50:00Z"
    },
    "upload": {
      "method": "PUT",
      "url": "期限付きアップロードURL",
      "headers": {
        "Content-Type": "video/mp4"
      },
      "expires_at": "2026-07-16T12:00:00Z"
    }
  }
}

```

ResponseへOriginalObjectKeyを含めない。

Upload完了通知

Request

Bodyは不要とする。

Object KeyをClientから受け取らない。

Response

```

{
  "id": 125,
  "processing_status": "uploaded",
  "publish_status": "private"
}

```

HTTPは **202 Accepted** とする。

同じ通知の再送時もJobを増やさず、現在状態を返す。

公開リール

```

{
  "items": [
    {
      "id": 125,
      "title": "ハンドドリップの蒸らし方",
      "description": "蒸らし時間の違いを紹介します",
      "category": "brewing",
      "author": {
        "id": 10,
        "name": "山田 太郎"
      },
      "playback_url": "期限付き再生URL",
      "thumbnail_url": "期限付きサムネイルURL",
      "is_saved": false,
      "created_at": "2026-07-16T11:50:00Z"
    }
  ],
  "next_cursor": null,
  "has_more": false
}

```

自分の投稿詳細

非同期動画処理の失敗情報

動画条件違反はWorkerで非同期に検出する。

投稿開始APIやUpload完了通知APIのHTTP処理中には、実ファイルの動画時間、縦横比、fps、破損状態を確定できない。

そのため、動画条件違反をUpload完了通知のHTTP Statusとして返さない。

自分の投稿詳細APIは、ProcessingStatusが `failed` の場合に安全な `failure_code` を返す。

failure_code

failure_code	内容
<code>invalid_format</code>	対応外の動画形式
<code>video_corrupt</code>	動画を解析できない
<code>duration_exceeded</code>	動画時間が上限を超えている
<code>size_exceeded</code>	ファイル容量が上限を超えている
<code>resolution_exceeded</code>	解像度が上限を超えている
<code>invalid_aspect_ratio</code>	縦横比が9:16ではない
<code>frame_rate_exceeded</code>	fpsが上限を超えている
<code>video_track_missing</code>	映像Trackが存在しない
<code>processing_failed</code>	利用者へ詳細を公開しない内部処理失敗

Responseルール

- ProcessingStatusが `failed` 以外の場合、`failure_code` は `null` とする。
- LastErrorMessageをAPI Responseへ返さない。
- FFmpegの標準出力や標準エラーを返さない。
- Storage SDKのエラーを返さない。
- SQLやStack Traceを返さない。
- 自分の投稿詳細API自体は `200 OK` で現在状態を返す。

```
{
  "id": 125,
  "title": "ハンドドリップの蒸らし方",
  "description": "蒸らし時間の違いを紹介します",
  "category": "brewing",
  "processing_status": "ready",
  "publish_status": "published",
  "failure_code": null,
  "playback_url": "期限付き再生URL",
  "thumbnail_url": "期限付きサムネイルURL",
  "created_at": "2026-07-16T11:50:00Z",
}
```

```
"updated_at": "2026-07-16T11:55:00Z"
}
```

Worker内部のLastErrorMessageは返さない。

動画アクセスの秘匿性

対象	取得ルール
元動画	公開URLにしない
元動画取得	Worker内部処理に限定し、FrontendへURLを返さない
ready / published	一般取得APIで期限付き再生URLを返せる
ready / private	投稿者本人用APIだけ期限付き再生URLを返せる
ready / hidden	一般取得APIおよび投稿者本人用APIでは再生URL・サムネイルURLを返さない。管理者投稿詳細APIで確認条件を満たす場合だけ管理者確認用の期限付きURLを発行できる。
削除済み	誰にも再生URLを返さない
Object Key	API Responseへ返さない
Upload URL	投稿開始Responseでだけ返す
Storage Credential	API、ログ、DBへ保存しない

動画の秘匿性は認証情報と同等に扱い、private・hidden・削除済み動画のURLを推測可能な固定公開URLにしない。

hidden動画について本編で確定すること

- ・ リールへ表示しない。
- ・ 保存一覧へ表示しない。
- ・ 後続の検索結果へ表示しない。
- ・ 投稿者は **published** へ再公開できない。
- ・ 投稿者用画面では **hidden** 状態であることを確認できる。
- ・ 投稿者用画面ではhidden状態を確認できるが、動画本体の再生URLは返さない。

Worker仕様

必須設定

設定は使用するプロセスごとに検証する。

APIはAPIが使用する設定だけを検証し、WorkerはWorkerが使用する設定だけを検証する。
Frontend設定が未設定であることを理由に、APIまたはWorkerを停止しない。

API設定

設定	用途
Upload URL有効期間	UploadExpiresAtと署名付きUpload URLの期限
IdempotencyRecord保持期間	ExpiresAt算出
Idempotency-Key HMAC鍵	KeyHash生成
Idempotency-Key最大長	Headerの過大入力拒否
投稿開始Rate Limit	投稿連打防止
Upload完了通知Rate Limit	完了通知連打防止
Object Storage接続設定	Upload URL、Read URL、Statの実行
Storage管理Prefix	Object Key生成先の限定

Worker設定

設定	用途
Worker同時実行数	同時FFmpeg実行上限
動画処理Job取得間隔	Processing Job Polling
Storage削除Job取得間隔	Cleanup Job Polling
Upload期限切れ回収間隔	Videoのexpired化
動画処理Job異常回収間隔	running Processing Job回収
Storage削除Job異常回収間隔	running Cleanup Job回収
孤立Object検出間隔	Storage残骸確認
孤立Object最小経過時間	Upload直後Objectの誤判定防止
孤立Object一回取得件数	Storage一覧の分割取得
IdempotencyRecord削除間隔	期限切れRecord削除
IdempotencyRecord一回削除件数	期限切れRecordの分割削除
動画処理再試行待機時間一覧	1回目、2回目、3回目のAvailableAt算出
Storage削除再試行待機時間一覧	Cleanup Jobの1回目、2回目、3回目のAvailableAt算出
Object Storage接続設定	Download、Upload、Delete、Listの実行
Storage管理Prefix	孤立Object検出範囲の限定

Frontend設定

設定	用途
API Base URL	Backend APIへの接続
Frontend Polling間隔	投稿状態の再取得
Object Storage Upload許可Header	署名付きURLへ送るHeaderの制御

固定値

設定	値
動画処理Timeout	5分

設定	値
VideoProcessingJob MaxAttempts	4
StorageCleanupJob MaxAttempts	4

動画処理時間ログ

項目	記録内容
Job識別	<code>job_id</code> 、 <code>video_id</code> 、 <code>attempt</code>
Stage	<code>claim</code> 、Download、Probe、Transcode、Thumbnail、Output Probe、Upload、DB確定など処理Stage
Stage状態	<code>started</code> 、 <code>completed</code> 、 <code>failed</code>
Stage処理時間	<code>duration_ms</code>
Job開始からの経過	<code>elapsed_ms</code>
Job取得	<code>queue_wait_ms</code> 、 <code>job_age_ms</code>
失敗	安全な <code>failure_code</code> 、 <code>retryable</code>
Summary	最終Statusと <code>total_ms</code>

Token、Cookie、Storage Credential、Object Key、署名付きURL、FFmpegコマンド全文を処理時間ログへ出力しない。

起動時検証

- APIはAPI設定が未設定、不正、または0以下の場合に起動しない。
- WorkerはWorker設定が未設定、不正、または0以下の場合に起動しない。
- FrontendはFrontend設定が不正な場合にBuildまたは起動を失敗させる。
- HMAC鍵やStorage Credentialをログへ出力しない。
- 再試行待機時間一覧は3件であることを確認する。
- 一回取得件数と一回削除件数は1以上であることを確認する。

APIとWorkerは責務上分離して実装する。現在の本番環境では費用面を考慮してAPIとWorkerを同一Render Service上で動作させる。負荷増加時にはWorkerを独立Serviceへ分離できる構成を維持する。

Frontend仕様

URL

URL	画面	認証
<code>/</code> または <code>/reels</code>	リール	不要
<code>/videos/:video_id</code>	動画詳細	不要
<code>/videos/author/:author_id</code>	投稿者ごとの公開動画一覧	不要

URL	画面	認証
<code>/videos/upload</code>	動画投稿	必須
<code>/me/videos</code>	自分の投稿	必須
<code>/me/saved-videos</code>	保存一覧	必須

動画投稿画面

入力

- Title
- Description
- Category
- 動画ファイル

Client事前検証

- MP4またはMOV
- 50,000,000 bytes以下
- 10秒以下
- 9:16
- 1080×1920以下
- 60fps以下はブラウザで正確に取得できない場合があるため、サーバー検証を最終判定とする

投稿処理

1. 投稿ボタン押下時にIdempotency-Keyを生成する。
2. 通信再送では同じKeyを使う。
3. 入力変更後の新規投稿では新しいKeyを作る。
4. 投稿開始APIを呼ぶ。
5. Upload URLへ動画をPUTする。
6. Upload進捗を表示する。
7. 完了後にUpload完了通知を送る。
8. 自分の投稿画面へ遷移する。
9. 状態が終端になるまで制御された間隔で再取得する。

二重クリック中は同じ投稿開始処理を複数起動しない。

投稿状態Polling

1. Upload完了通知成功後、自分の投稿詳細APIを設定された間隔で取得する。
2. ProcessingStatusが `uploading`、`uploaded`、`processing` の場合はPollingを継続する。

3. ProcessingStatusが `ready` の場合はPollingを停止する。
4. ProcessingStatusが `failed` の場合はPollingを停止する。
5. ProcessingStatusが `expired` の場合はPollingを停止する。
6. 投稿削除APIが成功した場合は即座にPollingを停止する。
7. 自分の投稿詳細APIが404を返した場合はPollingを停止して一覧へ戻す。
8. `deleted` をProcessingStatusとして扱わない。
9. ComponentがUnmountされた場合はPollingを停止する。
10. 前回Requestが完了していない場合は次のRequestを送信しない。
11. Browser Tabが非表示の場合は不要な高頻度Pollingを行わない。

Polling間隔は設定値としてFrontendへ定義する。

要件にない数値を仕様書内へ直接固定しない。

リール画面

- 縦スクロールで1件ずつ表示する
- 現在表示中の動画だけ再生する
- 画面外へ移動した動画を停止する
- 次の1件だけ先読みする
- 多数の動画本体を同時取得しない
- autoplay時はブラウザ制約を考慮してmutedで再生可能にする
- 表示中の動画について、利用者が音声をON/OFFへ切り替えられる操作を表示する
- 音声切り替えは現在表示中の動画に対して行い、複数動画を同時再生しない
- 再生失敗時にページ全体を停止しない
- 次Cursorを使って追加取得する
- has_moreがfalseなら追加取得しない
- 動画の投稿者名から `/videos/author/:author_id` の投稿者別公開動画一覧へ移動できる

動画詳細画面

表示するもの。

- 投稿者
- タイトル
- 説明
- カテゴリー
- 動画
- サムネイル

- 保存状態
- お気に入りへの保存・保存解除操作
- HTMLとして実行せず、Reactの通常の文字列出力を使用する。
- 投稿者名を選択すると `/videos/author/:author_id` へ移動し、その投稿者の公開動画一覧を確認できる。

投稿者ごとの公開動画一覧画面

- URLは `/videos/author/:author_id` とする。
- Route Parameterの `author_id` を取得し、既存のGET `/videos?author_id=:author_id`を使用する。
- `ready / published / deleted_at IS NULL`の動画だけを表示する。
- `private`、`hidden`、処理未完了、削除済み動画を表示しない。
- Cursorで追加取得する。
- 動画を選択すると既存動画詳細へ移動する。
- フォロー、プロフィール編集など、本要件にない機能は追加しない。

自分の投稿一覧

表示するもの。

- タイトル
- カテゴリー
- ProcessingStatus
- PublishStatus
- 投稿日時
- サムネイル
- 詳細への導線

状態表示例：

状態	表示
uploading	アップロード中
uploaded	処理待ち
processing	動画処理中
ready / published	公開中
ready / private	非公開
ready / hidden	管理者により非公開
expired	アップロード期限切れ
failed	動画処理失敗

非公開・再公開・削除

- published時だけ投稿者非公開を表示する
- privateかつready時だけ再公開を表示する
- hidden時は再公開ボタンを表示しない
- 削除済みは操作させない
- 送信中はボタンを無効化する
- 409時は詳細を再取得する
- 削除成功後は一覧から除外する

お気に入り保存UI

- 公開動画の保存操作は画面上で「お気に入りに保存」と表示する。
- 保存操作に使用するアイコンはヘッダーで使用する保存アイコンと統一する。
- 保存済みの場合は現在状態が分かる表示にする。
- 保存・解除APIやDB上のSavedVideo

保存一覧

- 自分が保存した動画だけを表示する
- 現在も公開条件を満たす動画だけ表示する
- hidden、private、deletedは表示しない
- 保存解除後は一覧から削除する
- Cursorで追加取得する

OpenAPI仕様

OpenAPIには次を必ず記載する。

- Bearer認証の有無
- Optional Authの意味
- Idempotency-Key必須
- Request Body
- Video ID Path Parameter
- limit、cursor
- 成功Response
- 400、401、403、404、409、413、429、500
- Upload完了が202であること

- Object Keyを返さないこと
- private・hidden・deletedへ一般再生URLを返さないこと
- 同一Idempotency-Key再送時の挙動
- 同一Key・異なるRequest時は409になること
- Upload完了通知の再送でJobを増やさないこと
- GET /videosの任意Query author_id author_id
- 指定時も公開条件を維持すること
- author_idが不正な場合は400であること
- 投稿者別一覧でも既存の公開動画一覧Responseを使用すること
- private、hidden、削除済み、処理未完了動画を返さないこと
- Cursorがauthor_idの異なる条件へ使い回されないこと

descriptionは実装条件が分かる文章にする。

認証済みのactiveユーザーについて、動画投稿情報を `uploading / private` で作成し、元動画を Object Storageへ直接送信するための期限付きUpload URLを返す。同じIdempotency-Keyと同じ入力が再送された場合は新しいVideoを作成せず、既存Videoを返す。

OpenAPIへ次を追加

- `failure_code` の定義
- `failure_code` は自分の投稿詳細でだけ返すこと
- ProcessingStatusが `failed` 以外の場合は `null` であること
- 動画条件違反は非同期処理結果として確認すること
- Worker内部のLastErrorMessageを返さないこと

エラー仕様

HTTP	条件
<code>400 Bad Request</code>	JSON、Path、Query、Cursor、Idempotency-Keyの形式不正
<code>401 Unauthorized</code>	未認証、不正Access Token、TokenVersion不一致、suspended User
<code>403 Forbidden</code>	他人の投稿を操作しようとした
<code>404 Not Found</code>	Videoが存在しない、または一般取得対象外
<code>409 Conflict</code>	状態競合、冪等性Key競合、Upload期限切れ、元Object未確認
<code>413 Content Too Large</code>	JSON Body Limit超過
<code>429 Too Many Requests</code>	動画投稿開始またはUpload完了通知のRate Limit超過
<code>500 Internal Server Error</code>	DB、Storage、署名URL発行、内部処理失敗

APIエラー形式は共通形式を使用する。

```
{
  "status": 409,
  "code": "video_state_conflict",
  "message": "動画の現在状態ではこの操作を実行できません",
  "request_id": "...
}
```

返してはいけないもの。

- SQL
- 制約名
- Stack Trace
- FFmpegコマンド全文
- Storage Credential
- Object Key
- 署名付きURL
- Password
- Access Token
- Refresh Token
- Cookie
- HMAC鍵

セキュリティ仕様

リスク	対策
XSS	Title、Description、NameをHTMLとして実行しない
CSRF	Bearer認証の動画APIにはRefresh Cookieを使わない
SQL Injection	<code>author_id</code> を含むQuery入力をSQL文字列へ直接連結せず、GORMプレースホルダーを使用する。ORDER BYは固定する。
Command Injection	Shellを使わず、固定引数を個別指定
Path Traversal	Userファイル名を作業パス・Object Keyに使わない
不正Upload	保存先、Content-Type、期限を署名URLで限定する
Object上書き	サーバー生成の一意Object Keyを使う
MIME偽装	FFprobeと実体MIME判定を使用する
重複投稿	Idempotency Record
重複変換	1Video1Job、行ロック、SKIP LOCKED
無限再試行	AttemptCountとMaxAttempts
Worker占有	5分Timeout
CPU・Memory枯渇	Worker同時実行上限

リスク	対策
Storage残骸	Cleanup Job、補償削除、孤立Object確認
秘密情報流出	Object Key、Credential、署名URLをResponse・Logへ出さない
他人の投稿操作	DBのUserIDと認証User IDを照合する

テスト仕様

Entity

- Video初期状態がuploading/private
- 不正な状態遷移を拒否する
- Upload期限切れ後にCompleteUploadできない
- SourceMeta二重設定を拒否する
- 処理成功時にactive Userならpublished
- suspended Userならprivate
- hiddenを投稿者が再公開できない
- DeletedAt設定後に再公開できない
- SourceVideoMetaの全境界値
- OutputVideoMetaの全境界値
- JobのAttemptCountが4を超えない
- SavedVideoの重複を拒否する
- Idempotency Key競合を判定する
- Cleanup Jobの対象なしを成功扱いにする
- IdempotencyRecordのExpiresAtがCreatedAtより後になる
- ExpiresAtが不正なIdempotencyRecordを生成できない
- SourceMeta設定済みの場合に二重保存しない
- 再試行時にVideoが `processing / private` のまま維持される
- `deleted` をProcessingStatusとして扱わない

Repository

- VideoとIdempotencyRecordが同時Commitされる
- 途中失敗で両方Rollbackされる
- 完了通知の同時実行でJobが1件になる
- ClaimNextで同じJobを複数Workerが取れない
- Job Claim Transactionが長時間保持されない

- 公開一覧にprivate、hidden、deletedを含めない
- 自分の投稿は他UserのVideoを含めない
- Save重複で関係行が増えない
- 保存一覧に非公開Videoを含めない
- Delete時にCleanup Jobが同時作成される
- Cursor境界で重複・欠落しない
- User入力をSQLへ連結しない
- queued Job取得時にJobとVideoが同じTransactionで更新される
- retry_wait Job取得時にVideoを再度StartProcessingしない
- ClaimNext途中失敗時にJobだけrunningとして残らない
- SourceMetaが未設定の場合だけ保存される
- SourceMeta設定済み時に重複作成されない
- running Job回収時にJobとVideoが同じTransactionで更新される
- Upload期限切れ時にVideoとCleanup Jobが同じTransactionで更新される
- 孤立Objectの未完了Cleanup Jobを重複作成しない
- 期限切れIdempotencyRecordだけを削除する
- author_id指定時に対象投稿者の公開動画だけを返す
- 他の投稿者の動画を返さない

対象投稿者でもprivate、hidden、deleted、処理未完了動画を返さない
author_id条件を維持したCursor追加取得で重複・欠落しない

Usecase

- 投稿開始でObject Keyをサーバー生成する
- 同一Key・同一入力で同じVideoを返す
- 同一Key・異なる入力で409相当のDomain Error
- Upload期限切れ処理
- Object不存在時にJobを作らない
- 完了通知再送時にJobを増やさない
- 条件違反動画を再試行しない
- 一時Storage障害を再試行する
- 最大4回で最終失敗する
- User停止中は自動公開しない
- hidden動画を投稿者再公開できない

- 削除時にObjectごとのCleanup Jobを作成する
- author_idを公開一覧条件としてRepositoryへ渡す
- author_idの異なる条件でCursorを使い回さない
- 投稿者別一覧でも期限付き再生URL・サムネイルURLを公開動画だけへ発行する

Controller

- 不正JSON
- Idempotency-Keyなし
- 不正Video ID
- 不正Cursor
- Usecase ErrorのHTTP変換
- 201、202、200、204
- 400、401、403、404、409、413、429、500
- Request IDをエラーへ含める
- Object KeyをResponseへ含めない
- author_idを正しくQueryから取得する

不正author_idを400へ変換する

author_idに未指定時は通常の公開一覧として処理する

Worker

- FFprobe成功
- FFprobe解析不能
- MIME偽装
- 10秒境界
- 50,000,000 bytes境界
- 9:16判定
- 回転メタデータ適用後の縦横判定
- 60fps境界
- 無音動画
- 変換後動画がMP4 / H.264 / 720×1280 / 30fps以下を満たす
- 音声ありの場合AACとなる
- 無音動画では映像のみのMP4となる
- Web配信用軽量化後もOutputVideoMetaの公開条件を満たす
- AAC変換

- FFmpeg Timeout
- 一時障害再試行
- 恒久障害即失敗
- running Job異常回収
- 一時ディレクトリ削除
- 同時実行上限
- 削除済みVideoを公開しない
- Upload期限切れ回収Loopが実行される
- running動画処理Job回収Loopが実行される
- running Cleanup Job回収Loopが実行される
- 孤立Object検出Loopが実行される
- IdempotencyRecord削除Loopが実行される
- 一つのLoop失敗でWorker全体が終了しない
- Loop間隔が0以下の場合に起動を拒否する
- Job取得時にqueue待機時間を計測できる
- 主要Stageのduration_msとelapsed_msを記録する
- 失敗時に安全なfailure_codeとretryableを記録できる
- 処理終了時に総処理時間をSummaryとして記録できる

Frontend

- 投稿入力検証
- 投稿ボタン二重送信防止
- Upload進捗表示
- Upload失敗表示
- 完了通知再送
- 状態Polling停止条件
- 表示中動画の音声をON/OFFへ切り替えられる投稿者名を選択すると/videos/author/:author_idへ遷移する
- AuthorVideosPageが指定投稿者の公開動画を表示するAuthorVideosPageがCursorで追加取得できる
- 保存操作に「お気に入りに保存」と表示する
- 保存・解除後に現在状態が反映される
- hidden時に再公開ボタンを出さない
- Title・Description内のHTMLを実行しない

- `ready` でPollingを停止する
 - `failed` でPollingを停止する
 - `expired` でPollingを停止する
 - 削除成功でPollingを停止する
 - 詳細APIの404でPollingを停止する
 - `deleted` をProcessingStatusとして判定しない
 - `failure_code` に応じた安全な表示を行う
 - Worker内部Error Messageを表示しない
-

API結合テスト

正常投稿

1. UserでLoginする。
2. 投稿開始APIを呼ぶ。
3. Videoがuploading/privateで作成される。
4. Storageへ直接Uploadする。
5. 完了通知を呼ぶ。
6. Videoがuploadedになる。
7. Processing Jobが1件できる。
8. Workerがprocessingへ変更する。
9. 検証・変換・サムネイル生成が成功する。
10. Jobがsucceededになる。
11. Videoがready/publishedになる。
12. リールへ表示される。

投稿者別公開動画一覧

複数Userで公開動画を作成する。

1. `GET /videos?author_id=<対象User ID>` を呼ぶ。
2. 対象Userの公開動画だけが返る。
3. 他Userの動画が返らない。
4. 対象Userのprivate動画を返さない。
5. hidden動画を返さない。
6. 削除済み動画を返さない。
7. 処理未完了動画を返さない。

8. Cursor追加取得でも同じ `author_id` 条件が維持される。

冪等性

1. 同じIdempotency-Keyで投稿開始を2回送る。
2. Videoが1件だけ作成される。
3. 異なる入力を同じKeyで送る。
4. 409になる。
5. 完了通知を複数回送る。
6. Processing Jobが1件だけ存在する。

条件違反

1. 10秒超過動画をUploadする。
2. WorkerがFFprobeで検出する。
3. 変換へ進まない。
4. Videoがfailed/privateになる。
5. リールへ表示されない。

停止User

1. Upload後、処理完了前にUserをsuspendedにする。
2. Worker処理を成功させる。
3. Videoがready/privateになる。
4. リールへ表示されない。
5. User再開後も自動公開されない。

削除

1. ready/published動画を削除する。
2. DeletedAtが設定される。
3. privateになる。
4. リール、詳細、保存一覧から消える。
5. Cleanup Jobが3種類作成される。
6. WorkerがStorageから削除する。

ブラウザ確認

投稿

1. Loginする。
2. 投稿画面を開く。
3. Title、Description、Categoryを入力する。
4. 条件内動画を選ぶ。
5. 投稿する。
6. Upload進捗が表示される。
7. APIサーバーへ動画本体が送られていないことを確認する。
8. 自分の投稿画面へ移動する。
9. uploading、uploaded、processing、readyの変化を確認する。
10. 公開動画がリールへ表示される。

閲覧

1. Logout状態でリールを開く。
2. 公開動画だけ表示される。
3. 表示中動画だけ再生される。
4. 音声ON/OFFを切り替えられる。
5. 次の動画へスクロールする。
6. 前の動画が停止する。
7. 詳細画面へ移動する。
8. 投稿者、Title、Description、Categoryが表示される。
9. Login後、投稿者名を選択する
10. `/videos/author/:author_id` へ移動する。
11. 選択した投稿者の公開動画だけが表示される。

保存

1. Loginする。
2. 公開動画の「お気に入り保存」を操作する。
3. 保存状態が画面へ反映される。
4. 再度保存してもDB上の保存関係が重複しない。
5. 保存一覧へ表示される。
6. 保存解除する。
7. 保存一覧から消える。
8. 保存操作のアイコンがヘッダーと同じデザインであることを確認する。

投稿管理

1. 自分の公開動画を非公開にする。
2. リールから消える。
3. 自分の投稿一覧ではprivateとして確認できる。
4. 再公開する。
5. リールへ戻る。
6. 管理者によりhiddenとなった動画では再公開操作が表示されない。
7. 投稿を削除する。
8. 一般画面から見えなくなる。
9. Storageファイルが非同期削除される。

管理者・ユーザー管理編との横断接続

動画編完成後、管理者・ユーザー管理編へ次を接続する。

1. 管理者ユーザー詳細へ投稿動画一覧を追加する。
2. User停止Transaction内でready/published/未削除動画をhiddenへ変更する。
3. 変更したVideoごとに `video_hide_by_user_suspension` 監査ログを作成する。
4. private、hidden、processing、failed、deletedは変更しない。
5. 停止Userの処理成功時はready/privateとする。
6. User再開時にhidden動画を自動公開しない。
7. リール、投稿者ごとの公開動画一覧、保存一覧、検索からhidden動画を除外する。

FFmpeg設定の改善履歴

段階	Commit	主な設定変更
初期実装	<code>8f156193</code>	<code>preset=medium</code> 、720x1280、30fps、AAC
通信量制御	<code>72a1e7cc</code>	CRF 23、最大3Mbps、Buffer 6Mbps、Metadata削除
負荷軽減	<code>966457cb</code>	<code>preset=veryfast</code> 、Video Encoder 1 thread

重たい動画の検証条件

- 10秒
- 1080×1920
- 60fps
- H.264+AAC

- 47,312,729 bytes
- 各設定3回実行
- 経過時間・CPU時間は中央値
- Memoryは最大RSS

実際の比較結果

段階	出力容量	入力比削減	経過時間	CPU時間	最大RSS
初期実装	23,424,942 bytes	50.5%	4.621秒	28.638秒	482,844 KiB
通信量制御	4,124,534 bytes	91.3%	3.252秒	17.864秒	481,924 KiB
現行設定	4,214,913 bytes	91.1%	6.910秒	8.991秒	230,404 KiB

- 通信量制御により、初期実装比で出力容量を82.4%削減
- 現行設定は処理時間が増える
- 一方でCPU時間を49.7%削減
- 最大RSSを52.2%削減
- 小規模WorkerでCPU・Memoryを占有しないことを優先
- MP4／H.264／720×1280／30fps／AACは維持